VALUE ADDED PROBLEM 1:

Given a Binary Search Tree and a range [low, high]. Find all the numbers in the BST that lie in the given range. Note: Element greater than or equal to root go to the right side. Concept: Binary Search Tree

**Example 1:**

**Input:**

```
        17
      /    \
     4      18
   /   \
  2     9
l = 4, h = 24
```

**Output:**

4 9 17 18

**Example 2:**

**Input:**

```
        16
      /    \
     7      20
   /   \
  1     10
l = 13, h = 23
```

**Output:**

16 20

```
#include <stdio.h>

#include <string.h>

#include <stdbool.h>

#define MAX 1000000

char stack[MAX];

int top = -1;

void push(char c) {

   stack[++top] = c;}

char pop() {

   if (top == -1) return '\0';

   return stack[top--];

}bool isEmpty() {

   return top == -1;}

bool isMatch(char open, char close) {

   return (open == '(' && close == ')') ||

         (open == '{' && close == '}') ||

         (open == '[' && close == ']');}
```

```
bool isBalanced(char* s) {
    top = -1; // reset stack
    for (int i = 0; s[i] != '\0'; i++) {
        char ch = s[i];
        if (ch == '(' || ch == '{' || ch == '[') {
            push(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
             if (isEmpty()) return false;
            char top_char = pop();
            if (!isMatch(top_char, ch)) return false;
        }
    }
    return isEmpty();
}
int main() {
    printf("ARYAN SHEGOKAR , 25070521180 , SECTION C(2)\n\n");
 char* tests[] = {"[{()}]", "[()(){}", "([)", "[{()})"};
    int n = 4;


    for (int i = 0; i < n; i++) {
        printf("Input:  \"%s\"\n", tests[i]);
        printf("Output: %s\n\n", isBalanced(tests[i]) ? "true" : "false"); }


    return 0;
}
```

Output

```
ARYAN SHEGOKAR , 25070521180 , SECTION C(2)

Example 1 Output: 4 9 17 18
Example 2 Output: 16 20


=== Code Execution Successful ===
```